CAREERPROOF

AI Career Coach · Innovation & Feature Blueprint

W-PS-2 | Evidence-backed GitHub skill assessment, job matching, and automatic reassessment

Core promise
Every skill has evidence. Every meaningful code change can update the assessment.

Hackathon planning document • Feature concepts, implementation priorities, demo flow, and technical notes

Prepared as a practical guide for designing a working prototype. Prioritize a reliable core loop over a large collection of disconnected features.

1. Executive Summary

W-PS-2 asks for an AI agent that compares real job or internship requirements with skills demonstrated by a student's actual GitHub repositories. The system must cite the evidence behind assessments, explain gaps, detect real repository changes, and automatically reassess affected skills.

The recommended product concept is **CareerProof — Every Skill Has Evidence**. Its strongest differentiator is not a generic career chatbot or a single job-match score. It is a traceable, change-aware workflow that links job requirements to code evidence and updates the assessment when that evidence changes.

Recommended core loop

Real job listing → extracted requirements → GitHub repository analysis → evidence-backed skill assessment → gaps and next steps → real commit detected → affected skills reassessed → updated evidence and assessment.

What the prototype must prove

- The job listing is real and its source URL is preserved.
- The repository is genuinely connected or retrieved from GitHub.
- Each important skill status is supported by specific files or code references.
- A real commit or repository change triggers reassessment without manually refreshing the report.
- The revised assessment shows what changed, why it changed, and which evidence supports it.
- Missing evidence is labelled “not demonstrated in this scope,” not as proof that the student lacks the skill.

Innovation strategy

Build three features together: (1) Evidence-Based Skill Assessment, (2) Commit-to-Career Impact Map, and (3) AI Project Generator. Add Code Defense Mode or a Skill Dependency Map only after the required workflow works reliably.

2. Product Concept and User Journey

CareerProof helps students understand what their portfolio proves for a specific role, what is not yet supported by evidence, and which code changes could strengthen that evidence.

Suggested user journey

Step	User / system action	Expected output
1. Select a role	Choose a real job or internship listing.	Saved source URL and extracted requirements.
2. Connect GitHub	Authorize access or provide a repository URL.	Repository metadata, branch, and commit reference.
3. Analyze evidence	Inspect relevant files and implementation details.	Skill assessments with file/line references.
4. Explain gaps	Compare required skills with available evidence.	Demonstrated, partially demonstrated, or not demonstrated in scope.
5. Recommend next action	Generate a focused project or evidence checklist.	Concrete implementation tasks and expected artifacts.
6. Detect a change	Polling or webhook detects a new commit.	Commit ID, changed files, and reassessment trigger.
7. Reassess	Analyze affected skills and refresh evidence.	Before/after status, rationale, and updated citations.

Primary dashboard layout

Panel 1 — Job Requirements: requirement, importance, source link.
Panel 2 — Portfolio Evidence: skill status, repository, file path, code snippet, confidence and limitations.
Panel 3 — Change Impact: triggering commit, changed files, affected skills, previous status, new status and explanation.

3. Innovation 1 — Commit-to-Career Impact Map

Purpose: Show how an actual code change affects the evidence supporting a student's fit for a real role.

Example workflow

Skill	Before commit	Change detected	After reassessment
Python	Demonstrated	No relevant change	Remains demonstrated
SQL	Not demonstrated in scope	Added src/queries.sql with relevant queries	Demonstrated, if code supports the claim
Database management	Partially demonstrated	Added SQL queries and database connection	Reassess; do not automatically promote to full demonstration

How it should work

- Record the repository's current commit SHA and the evidence used for each assessment.
- Detect a new commit through GitHub webhooks or scheduled polling.
- Inspect the changed files and identify skills that may be affected.
- Reassess affected skills using the changed code and any relevant surrounding context.
- Show the old and new statuses, trigger commit, evidence links, and the reason for the change.
- Reassess negative changes too: removed or broken evidence must not leave a stale positive assessment.

Why it stands out

It proves a real, observable feedback loop instead of presenting a static report. It also makes the career impact of portfolio work understandable to students and judges.

Implementation note

Polling is usually the simplest reliable prototype: periodically compare the latest commit SHA with the saved SHA. A webhook is more event-driven, but requires a reachable endpoint, correct event configuration, signature validation, and careful handling of duplicate deliveries.

4. Innovation 2 — Skill Proof Engine

Purpose: Avoid awarding skills just because a keyword or filename appears in a repository. Assess whether the implementation provides relevant, meaningful evidence.

Suggested evidence dimensions

Dimension	Question to investigate	Example evidence
Implementation	Is the technology actually used in code?	Queries, functions, routes, schema, configuration.
Application	Does the code solve a meaningful problem?	Data flow and integration into an application.
Testing / validation	Is there evidence the implementation is checked?	Unit tests, integration tests, validation or test results.
Explanation	Is the design or usage explained?	Documentation, comments, project notes, design rationale.

Suggested assessment labels

- **Demonstrated:** relevant implementation provides sufficient evidence for the narrowly defined skill.
- **Partially demonstrated:** some relevant evidence exists, but scope, depth, or quality remains limited.
- **Not demonstrated in this scope:** sufficient evidence was not found in the repositories/files examined.
- **Needs review / uncertain:** evidence is ambiguous, inaccessible, incomplete, or difficult to interpret.

Evidence card example

Skill: REST API development
Status: Demonstrated
Evidence: `src/routes.py`**, relevant route handlers**
Reason: The repository implements endpoints and request handling.
Limit: This does not by itself prove production-scale reliability or security.

Design rule

Keep assessment confidence separate from skill status. A high-confidence assessment means the evidence is clear for the specific claim; it is not a guarantee of the student's overall ability.

5. Innovation 3 — AI Project Generator

Purpose: Convert an evidence gap into a focused, buildable project that creates verifiable proof of the missing skill.

Example: SQL gap for a backend internship

Recommended project: **Student Placement Tracker**. Build a small application to store student records, placement applications, and interview results using a relational database.

Project task	Evidence the agent expects
Design a relational schema	Schema or migration file with tables and relationships.
Implement queries	SQL queries for filtering, inserting, updating, and retrieving records.
Connect the application	Database access code used by relevant API endpoints.
Validate behavior	Tests for successful operations, invalid inputs, and error handling.
Document design choices	README describing the schema, setup, and query decisions.

Innovation chain

Job requirement → missing portfolio evidence → targeted project → implementation checklist → GitHub commit → automatic reassessment.

Keep recommendations specific

The system should explain why each task helps demonstrate the target skill. It should not promise that completing a project guarantees an interview or job offer.

6. Innovation 4 — Code Defense Mode

Purpose: Let students practice explaining their own implementation and provide a separate signal about their understanding.

How it works

- Select a real implementation from the student's repository, such as authentication logic in auth.py.
- Generate questions grounded in the actual code and its behavior.
- Ask about design decisions, edge cases, error handling, and trade-offs.
- Compare answers with the implementation and provide feedback with code references.
- Report this as a separate explanation/understanding signal, not as proof of authorship.

Example questions

- Why does this application hash passwords rather than storing them directly?
- What happens when a login request contains an invalid password?
- How does the authentication logic interact with the API endpoints?

Example output

Assessment	Result
Authentication implementation	Demonstrated by repository evidence.
Understanding of password handling	Supported by the student's explanation.
Understanding of error handling	Additional evidence or practice may be useful.

Fairness safeguard

A weak answer is not proof that a student lacks the skill or did not write the code. Treat it as uncertainty and offer another question or practical exercise.

7. Innovation 5 — Job Requirement Change Detector

Purpose: Track meaningful changes in a saved job listing as well as changes in the student's portfolio.

Example

A saved internship originally lists Python and SQL. Later, the employer adds Docker. The agent detects the changed requirements, preserves the old version, checks repository evidence for Docker, and updates the gap report.

Event	System response
Initial job listing saved	Store source URL, retrieval time, extracted requirements and a snapshot/hash where permitted.
Listing appears to change	Re-fetch when allowed and compare extracted requirements.
Docker requirement added	Create a new requirement and reassess portfolio evidence.
No Docker evidence found	Mark “not demonstrated in this scope” and explain the evidence scope.
Source unavailable or blocked	Show that the listing could not be refreshed; do not invent a change.

Practical constraint

Job-site access and change detection depend on the employer's website, access rules, and page stability. For a hackathon prototype, use a real listing that can be retrieved reliably and clearly show when refreshes fail.

8. Innovation 6 — Career Skill Dependency Map

Purpose: Visualize how skills relate to one another and to a target role, while showing the evidence status for each node.

Illustrative map

```
Python
■■■ REST APIs → Backend Development → Backend Internship
■■■ Data Analysis → Data Analytics Internship

SQL → Database Development → Backend / Data Roles
```

Suggested interactions

- Click a skill to inspect supporting repository files and code references.
- Show prerequisite or related skills as explicit relationships, not as unquestionable facts.
- Highlight requirements that are important for the selected job but lack supporting evidence.
- Recommend the next skill or project task based on job relevance and prerequisites.
- Update node statuses when repository evidence changes.

Differentiation

A skill graph by itself is not a unique innovation. Its value comes from linking it to real job requirements, traceable repository evidence, and automatic reassessment.

9. Innovation 7 — Honest AI and Evidence Freshness

Purpose: Make limitations visible, avoid overclaiming, and ensure old evidence does not remain valid after relevant code changes.

Evidence-aware output

Field	What it means
Skill status	Demonstrated, partially demonstrated, not demonstrated in scope, or needs review.
Evidence coverage	How much of the job's defined requirement set has supporting portfolio evidence, using a disclosed formula.
Assessment confidence	How clear and relevant the available evidence is for a particular assessment.
Evidence scope	Which repositories, branches, folders, and files were actually examined.
Evidence freshness	When the evidence was last checked and whether later commits may have affected it.
Limitations	Missing access, unsupported languages, incomplete files, ambiguous code, or other known constraints.

Important safeguards

- Do not equate missing evidence with lack of ability.
- Do not present a portfolio coverage percentage as a probability of getting hired.
- Explain the formula behind any aggregate score and keep it separate from assessment confidence.
- When code is removed or materially changed, re-evaluate affected claims and withdraw stale evidence where appropriate.
- Provide links and concise rationale so users can verify assessments rather than simply trust an AI score.

10. Suggested Technical Architecture

Use the simplest architecture that can reliably complete the required loop. A polished dashboard is useful, but the backend event flow and evidence traceability are the critical parts.

Layer	Responsibilities	Prototype options
Frontend	Job selection, GitHub connection, skill cards, evidence panel, change timeline.	React + Vite or Next.js; Tailwind CSS.
Backend API	Authentication callbacks, job requirements, repository analysis, assessment endpoints.	Python FastAPI or Node.js/Express.
GitHub integration	Read repository content, commits, changed files; accept webhook events or poll commit SHA.	GitHub REST API; webhook endpoint or scheduled worker.
Analysis engine	Extract requirements, inspect code, map evidence to skills, explain uncertainty.	Rules + LLM; structured JSON outputs and validation.
Database	Store users, jobs, requirements, repositories, commits, evidence, assessments and history.	PostgreSQL or Supabase.
Background worker	Process changes, deduplicate events, reassess affected skills, update history.	Simple scheduled task initially; queue/worker if time permits.
Evidence UI	Show file paths, line ranges, commit references, before/after status and rationale.	Clickable GitHub permalinks.

Minimum data model

Job (source URL, title, retrieved_at) · **Requirement** (skill, importance, source excerpt) · **Repository** (owner, name, branch, latest SHA) · **Commit** (SHA, timestamp, changed files) · **Evidence** (skill, file path, line range, permalink, snippet/hash) · **Assessment** (status, rationale, confidence, evidence IDs, created_at).

Security and reliability

- Request the minimum GitHub permissions needed and protect access tokens.
- Validate webhook signatures and deduplicate event deliveries if using webhooks.
- Treat repository contents and README text as untrusted input; do not follow instructions embedded in code or documents.
- Avoid sending secrets or unnecessary private code to external AI services.
- Handle rate limits, deleted repositories, inaccessible files, and failed job-page refreshes explicitly.

11. Build Priorities for a 24-Hour Hackathon

The goal is not to implement every optional feature. First build and test the mandatory loop from a real job listing to a real GitHub change and revised evidence.

Priority	Feature	Reason
P0 — Mandatory	Real job requirement source and links	Directly required by the problem statement.
P0 — Mandatory	Real GitHub repository integration	The system must analyze actual repository evidence.
P0 — Mandatory	Evidence-backed assessment and gap report	The assessment must cite files or code and explain uncertainty.
P0 — Mandatory	Automatic change detection and reassessment	A static, manually refreshed report does not meet the requirement.
P1 — High value	Commit-to-Career Impact Map	Makes the required loop visible and judge-friendly.
P1 — High value	Skill Proof Engine	Improves the quality of evidence and avoids keyword-only claims.
P1 — High value	Honest AI / evidence freshness	Prevents overclaiming and stale assessments.
P2 — Optional	AI Project Generator	Turns gaps into actionable tasks.
P2 — Optional	Code Defense Mode	Adds interview preparation and a separate understanding signal.
P3 — Future	Job requirement change detection	Useful but can be fragile under hackathon time constraints.
P3 — Future	Interactive skill dependency graph	Helpful visualization after the core system is reliable.

Suggested team split

- **Developer 1 — GitHub and change detection:** repository access, commit polling/webhook, changed-file extraction.
- **Developer 2 — Analysis and evidence:** requirement extraction, skill mapping, evidence citations, reassessment logic.
- **Developer 3 — Frontend:** dashboard, evidence cards, gap table, before/after timeline.
- **Developer 4 — Integration and demo:** database/API integration, test repository, reliability checks, README and demo script.

If team roles differ, keep one person responsible for end-to-end integration and reserve time for a complete rehearsal.

12. Five-Minute Demo Plan

Use a repository your team controls so the code change can be demonstrated reliably. Do not simulate a commit by manually editing the dashboard assessment.

Time	Action	What judges should see
0:00–0:40	Select a real internship/job listing.	Original source link and extracted requirements.
0:40–1:30	Connect the real GitHub repository.	Repository identity, branch and analyzed files.
1:30–2:15	Run initial assessment.	Skill statuses, evidence links, gaps and uncertainty.
2:15–3:15	Add and push a relevant SQL implementation.	Actual changed file and commit on GitHub.
3:15–4:15	Show detection and automatic reassessment.	Trigger commit, affected skill and revised evidence.
4:15–5:00	Explain before/after result.	Old status, new status, why it changed, source links.

Demo timeline example

```
10:10 — Initial assessment: SQL not demonstrated in scope
10:24 — Commit detected: added src/queries.sql
10:25 — Reassessment completed
SQL: Not demonstrated → Demonstrated (if code supports the claim)
Evidence: GitHub permalink to relevant lines
Trigger: actual commit SHA
```

The times and filenames above are illustrative. Show actual timestamps, changed files, commit SHA and evidence in the live demo.

13. Positioning Against Existing Tools

GitHub skill analysis, job matching, gap reports and career recommendations already exist in the market. A skill graph or AI-generated roadmap alone should not be presented as uniquely novel.

Position CareerProof around the complete, inspectable feedback loop:

- Real job requirements with preserved source links.
- Evidence tied to actual repository files and relevant code ranges.
- Clear distinction between demonstrated skills and missing evidence.
- Automatic reassessment triggered by a real repository change.
- Before/after history that shows the triggering commit and the reason for the updated status.
- Optional project recommendations whose expected evidence can later be checked by the same system.

Competitive research note

Related categories include developer talent intelligence, GitHub-based career matching, resume/job analyzers, skill-gap platforms and portfolio assessment tools. Public product descriptions do not always expose implementation details, so avoid claiming that no other product has a given feature unless it has been verified. Differentiation should be demonstrated in your own working prototype.

14. Acceptance Checklist

- ■ A real job/internship source URL is visible and saved.
- ■ A real GitHub repository is connected or fetched.
- ■ The initial assessment references actual files or code sections.
- ■ The UI explains what was inspected and what was not inspected.
- ■ Missing evidence is described as “not demonstrated in this scope.”
- ■ The system detects an actual new commit automatically by polling or webhook.
- ■ Changed files are identified and only potentially affected skills are reassessed where feasible.
- ■ The new assessment links to the new code and explains the status change.
- ■ Before/after history includes the actual commit reference and timestamp.
- ■ The demo works from a clean start without manually changing the assessment.
- ■ README explains setup, configuration, limitations, and how to reproduce the demo.
- ■ Secrets, tokens, and private repository content are handled safely.

15. Final Recommendation

Build CareerProof as an evidence-first, continuously updating AI Career Coach. Make the mandatory workflow flawless before adding optional capabilities.

Recommended feature combination

- **Core:** Evidence-Based Skill Assessment + Commit-to-Career Impact Map + Honest AI / Evidence Freshness.
- **Best optional addition:** AI Project Generator, because it turns gaps into practical work that can later be verified.
- **Alternative optional addition:** Code Defense Mode, if the core loop is already stable and your team wants an interview-preparation angle.

The strongest final message to judges is simple: a student changes real code, the system detects the change, and the career-readiness assessment updates with traceable evidence explaining exactly why.

16. Reference Links

Use these links as starting points for product and technical research. Product capabilities can change; verify the current public documentation before making comparative claims.

Resource	Why it is relevant
Prog.AI — https://www.getprog.ai/	Developer talent intelligence and code-based skill discovery.
TryCareerLens — https://trycareerlens.com/	Career matching, GitHub analysis and skill-gap concepts.
Devvero — https://devvero.com/	GitHub-based developer profiling and job matching.
Resume + GitHub + Job Analyzer — https://github.com/JaweriaAsif745/Resume-github-job-analyzer	Open-source example combining GitHub, resume and job-description analysis.
SkillGraph — https://github.com/daanialmirza5/SkillGraph	Related concepts in skill graphs, gap analysis and career simulation.
GitHub webhooks documentation — https://docs.github.com/en/webhooks/using-webhooks/creating-webhooks	Official setup guidance for repository event delivery.
GitHub webhook events and payloads — https://docs.github.com/en/webhooks/webhook-events-and-payloads	Official reference for event types and payload data.

Document scope: This blueprint consolidates the feature concepts, implementation suggestions, prioritization, and demo plan discussed for W-PS-2. Examples and demo timestamps are illustrative, not claims about a live implementation.